Week 1: Introduction & Progress Review

Roadmap, deliverables, grading, data sources, and resources

North Carolina State University | Summer 2026

NC STATE UNIVERSITY

Industry Mentor: Dendi Suhubdy (Bitwyre) | Guest: Fenni Kang (Coincall)

Contents

Learning Objectives

- Understand the arc of the program and what you will build
- Be clear on the timeline, the three milestones, and how you are graded
- Know where the data comes from (Coincall) and how to pull it
- Have the Python + PyTorch and C++/pybind11 toolchains running

Where We Are (Week 1 Progress Review)

- Maker vs. taker: we build the MAKER – continuously quoting both sides
- The spine of the course is the Avellaneda-Stoikov model
- Week 1 status: environment setup, data access, and this orientation
- By Week 5 you ship a calibrated quoting engine (Milestone 1)

Tooling & How to Get Help

- Problem sets: Python + PyTorch (autograd does differentiation for you)
- Fast computation (Week 7): C++ exposed to Python via pybind11
- Mentorship is asynchronous (email/Telegram); tutors run day-to-day
- Office hours with tutors; come with a minimal reproducible snippet

Timeline & Deliverable Clarification

- 10 weeks. Week 1 introduction (done). Weeks 2-10 technical.
- Milestone 1 (Wk 5): Avellaneda-Stoikov engine, Python/PyTorch
- Milestone 2 (Wk 7): fast pricing/Greeks library, C++ via pybind11
- Milestone 3 (Wk 9): full options market-making backtest
- Final (Wk 10): one advanced extension + report (<=30pp) + 20-min talk
- Problem sets are due Sunday 23:59 ET; milestones Friday of the week

Syllabus & Grading Walkthrough

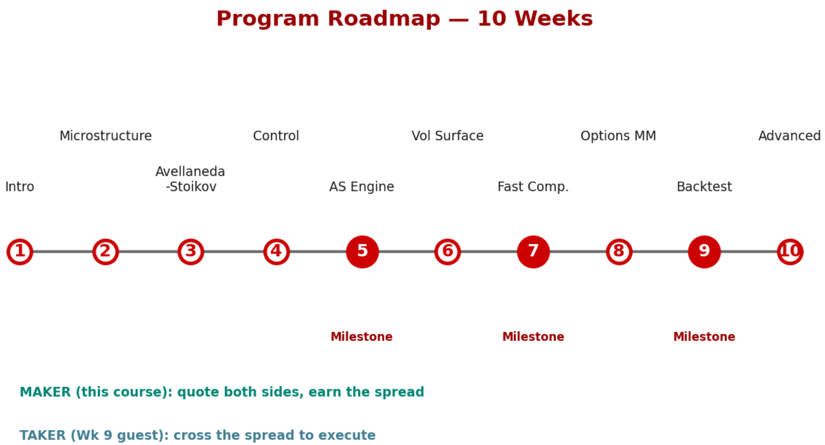
- Problem sets 30% | Milestones (3) 30% | Final 30% | Participation 10%
- Code graded on correctness, reproducibility, quality – in that order
- Numerical-verification problems: right setup + right error magnitude
- Milestone 2 latency is a HARD GATE (sub-5ms surface) – caps at 70 if missed
- Late PS: -20% until Tue; nothing after. Milestones: no late submission.
- Set random seeds: reproducibility is explicitly graded

Data Sources Discussion (Coincall)

- Exchange: Coincall. REST base: https://api.coincall.com
- Provided as Parquet: BTC/ETH options L2 + trades, perp L2, funding, SVI
- Coincall orderbook is L2 (aggregated depth) – there is NO L3 feed
- You can also pull live via the REST/WebSocket API (next slides)
- Auth: API key + HMAC-SHA256 signature; market data is largely public
- Always cross-check exact endpoints at https://docs.coincall.com/

Additional Resources – Math-Heavy Sections

- Stochastic control: Pham (2009), Ch. 3-4 (HJB, verification)
- Market making theory: Gueant (2016), Ch. 4; Cartea et al., Ch. 10-11
- Options/vol surface: Gatheral (2006); Gatheral-Jacquier (2014)
- Hedging: Taleb (1997), gamma scalping & the gamma-theta identity
- El Aoud-Abergel (2015): multi-Greek option market making (Week 8)
- We DERIVE the key results in lecture and explain every modeling choice


Program Roadmap — 10 Weeks
Microstructure
Control
Vol Surface
Options MM
Advanced
Intro
Avellaneda
-Stoikov
AS Engine
Fast Comp.
Backtest
1
2
3
4
5
6
7
8
9
10
Milestone
Milestone
Milestone
MAKER (this course): quote both sides, earn the spread
TAKER (Wk 9 guest): cross the spread to execute

Getting the Data from Coincall

- Base URL: https://api.coincall.com | Python SDK: github.com/CoincallExchange
- Signed request headers: X-CC-APIKEY, sign (HMAC-SHA256, upper hex), ts (ms), X-REQ-TS-DIFF
- Signing: sort params alphabetically, append uuid, ts, x-req-ts-diff, HMAC-SHA256 with secret
- Rate limits (per docs): orderbook 30/2s per user; public config 10/2s per IP
- Key endpoints you will use across the program:
  - `Instruments: GET /open/option/getInstruments/{baseCurrency}`
  - `Option chain: GET /open/option/get/v1/{index}`
  - `Option book: GET /open/option/order/orderbook/v1/{symbol} (100 depth)`
  - `Option detail: GET /open/option/detail/v1/{symbol} (mark, IV, delta/gamma/vega/theta)`
  - `Trades: GET /open/option/trade/lasttrade/v1/{symbol}`
  - `Futures book: GET /open/futures/order/orderbook/v1/{symbol}`
  - `Funding: GET /open/public/fundingRate/v1?symbol=BTCUSD`
  - `Klines: GET /open/option/market/kline/history/v1/{optionName}?period=h1`
- WebSocket channels: /options/pricing, /options/orderbook, /options/lasttrade, /options/kline
- ALWAYS verify the exact path/params against https://docs.coincall.com/ – the API evolves

Code — coincall_client.py

```
import time, hmac, hashlib, uuid, requests

BASE = "https://api.coincall.com"

def signed_get(path, params=None, api_key="", api_secret=""):
    """Minimal Coincall signed GET. Public market-data endpoints often work
    unsigned; this shows the signed pattern used for account endpoints."""
    params = params or {}
    ts, window = str(int(time.time() * 1000)), "5000"
    items = sorted(params.items()) # sort alphabetically
    payload = "&".join(f"{k}={v}" for k, v in items)
    payload += f"&uuid={uuid.uuid4().hex}&ts={ts}&x-req-ts-diff={window}"
    sign = hmac.new(api_secret.encode(), payload.encode(),
                    hashlib.sha256).hexdigest().upper()
    headers = {"X-CC-APIKEY": api_key, "sign": sign,
               "ts": ts, "X-REQ-TS-DIFF": window}
    r = requests.get(BASE + path, params=params, headers=headers, timeout=10)
    r.raise_for_status()
    return r.json()

# Example: list all BTC option instruments
print(signed_get("/open/option/getInstruments/BTC"))
```

Summary — Key Takeaways

- We build the MAKER side: continuously quote bid/ask and earn the spread
- Avellaneda-Stoikov is the spine; you ship a calibrated engine by Week 5
- Python/PyTorch for problem sets; C++ via pybind11 for fast computation
- Three milestones (Wk 5, 7, 9); grading is 30/30/30/10
- Data is Coincall (L2 only, no L3); confirm endpoints at docs.coincall.com

Glossary — Key Terms

Market maker	liquidity provider that continuously quotes bid and ask, earning the spread
Taker	participant that crosses the spread for immediate execution
Bid-ask spread	best ask minus best bid; the maker's revenue and risk premium
Inventory	the maker's net position; the main source of risk
Milestone	graded software deliverable, due in Weeks 5, 7, and 9
PyTorch / autograd	Python ML library; automatic differentiation computes Greeks for you
pybind11	library that exposes C++ functions to Python (used in Week 7)

Reading

- Cartea-Jaikumar-Penalva, Ch. 1; Avellaneda-Stoikov (2008) – skim
- Coincall API docs: https://docs.coincall.com/ (read 'Getting Started')

References

- Avellaneda, M., & Stoikov, S. (2008). High-frequency trading in a limit order book. Quantitative Finance, 8(3), 217-224.
- Gueant, O. (2016). The Financial Mathematics of Market Liquidity. Chapman & Hall/CRC.
- Cartea, A., Jaikumar, S., & Penalva, J. (2015). Algorithmic and High-Frequency Trading. Cambridge Univ. Press.
- Coincall API documentation. https://docs.coincall.com/